Support de Cours — Introduction à la POO en PHP

TP5 – Séance 1 | BTS SIO – Bloc 2

> 📋 Ce document est votre référence pour la séance 1. Il couvre les fondamentaux du paradigme objet : classe, objet, propriétés, méthodes et le mot-clé `$this`. La visibilité, les constructeurs et l'encapsulation sont traités en séance 2.

Table des matières

1. Pourquoi la POO ?
2. Les concepts fondamentaux
3. Modéliser avant de coder
4. Créer une classe en PHP
5. Instancier et utiliser un objet
6. Le mot-clé `$this`
7. Les méthodes en détail
8. Erreurs courantes
9. Exercices corrigés
10. Résumé

---

1. Pourquoi la POO ?

Les limites du procédural

Jusqu'à présent, vous avez utilisé la **programmation procédurale** : des variables pour stocker des données, des fonctions pour les traiter. Cette approche fonctionne bien pour des programmes simples, mais elle montre ses limites dès que la complexité augmente.

Le problème fondamental du procédural : **les données et les traitements sont déconnectés**. Rien n'empêche d'appeler une fonction sur des données qui ne lui correspondent pas, ou de modifier une variable par accident depuis n'importe où dans le code.

Ce que propose la POO

La Programmation Orientée Objet propose une autre façon d'organiser son code : **regrouper les données et les comportements dans une même entité logique**, appelée un **objet**.

Plutôt que d'avoir une variable `$solde` quelque part et une fonction `deposer()` quelque part ailleurs, on crée un objet `CompteBancaire` qui contient à la fois le solde *et* les actions possibles sur ce solde.

> 💡 La POO ne remplace pas les concepts que vous connaissez. Les variables deviennent des **propriétés**, les fonctions deviennent des **méthodes**. La logique reste la même — elle est simplement mieux organisée.

Ce que ça change concrètement

Ce qu'on faisait	Ce qu'on fait maintenant
Variables séparées dans le code	Données regroupées dans un objet
Fonctions qui manipulent des données	Méthodes qui appartiennent à un objet
Données accessibles et modifiables partout	Données encapsulées dans leur objet
Code difficile à réutiliser	Classes réutilisables dans différents contextes

---

2. Les concepts fondamentaux

L'objet

Un objet est une **entité qui combine des données et des comportements**. C'est la représentation logicielle d'une chose du monde réel ou d'un concept abstrait.

Un objet a deux composantes :

- **Ses propriétés** : les données qu'il contient, son état (ex : le nom d'un utilisateur, le solde d'un compte)
- **Ses méthodes** : les actions qu'il peut effectuer (ex : se connecter, déposer de l'argent)

Exemple — Objet « Utilisateur » :

Propriétés	Méthodes
nom	seConnecter()
email	seDeconnecter()
age	modifierProfil()
statut	envoyerMessage()

Toutes ces données et actions sont **regroupées dans un seul objet cohérent**. C'est l'essence de la POO.

La classe

Une classe est un **modèle**, un **plan** qui décrit comment seront construits les objets.

Analogie : Le plan d'architecte décrit comment construire une maison. Ce n'est pas la maison elle-même, mais à partir de ce plan, on peut construire autant de maisons qu'on veut. Chaque maison construite sera indépendante : l'une sera peinte en rouge, une autre en blanc ; l'une aura 3 chambres, l'autre en aura 4.

C'est exactement la relation entre une classe et ses objets :

- La **classe** `Voiture` décrit ce qu'est une voiture (elle a une marque, un modèle, elle peut démarrer, accélérer)
- Les **objets** sont les voitures concrètes : la Peugeot 308 de Marie, la Renault Clio de Paul

La relation classe → objet

```
Classe "Voiture" (le plan)
         │
         ├──→ $voiture_marie  →  marque: Peugeot, modèle: 308, couleur: bleue
```

```
        ├──→ $voiture_paul   →  marque: Renault, modèle: Clio, couleur: rouge
        └──→ $voiture_lucas  →  marque: Citroën, modèle: C3,   couleur: grise
```

Chaque objet est **indépendant**. Modifier les données d'un objet ne touche pas aux autres.

Les termes à maîtriser

Terme	Définition
Classe	Le plan, le modèle qui définit la structure des objets
Objet	Une instance concrète créée à partir d'une classe
Instance	Synonyme d'objet — résultat de l'instanciation
Propriété	Une variable appartenant à un objet
Méthode	Une fonction appartenant à un objet
Instanciation	L'acte de créer un objet à partir d'une classe

---

3. Modéliser avant de coder

Avant d'écrire du PHP, il faut **réfléchir à la modélisation** : quelles sont les données et les comportements de l'entité qu'on veut représenter ?

Démarche de modélisation

Pour modéliser un objet, posez-vous deux questions :

1. **Qu'est-ce que cet objet sait ?** → les propriétés
2. **Qu'est-ce que cet objet peut faire ?** → les méthodes

Exemples de modélisation

Compte bancaire :

Propriétés	Méthodes
titulaire	déposer($montant)
solde	retirer($montant)
numéro de compte	afficher le solde()

Étudiant :

Propriétés	Méthodes
nom, prénom	ajouterNote($note)
notes (tableau)	calculerMoyenne()
classe	obtenirMention()

Produit (e-commerce) :

Propriétés	Méthodes
nom	estDisponible()
prix	calculerPrixTTC()
stock	afficherInfo()

> 📌 Cette étape de réflexion est aussi importante que le code lui-même. En entreprise, on modélise sur un schéma (diagramme de classe UML) avant de commencer à coder.

---

4. Créer une classe en PHP

Structure de base

```php
<?php
class NomDeLaClasse {

    // Les propriétés (les données)
    public $propriete1;
    public $propriete2;

    // Les méthodes (les comportements)
    public function uneMethode() {
        // code de la méthode
    }
}
```

Pour cette séance, toutes les propriétés et méthodes sont déclarées en `public`. La signification de ce mot-clé et les alternatives (`private`, `protected`) seront étudiées en séance 2.

Conventions de nommage

Ces conventions sont des **standards professionnels** à respecter absolument.

Les classes → **PascalCase** : première lettre de chaque mot en majuscule.

```
✅  Voiture    CompteBancaire    GestionnaireBdd    PanierAchat
❌  voiture    comptebancaire    gestionnaire_bdd   panier_achat
```

Les propriétés et méthodes → **camelCase** : première lettre en minuscule, puis majuscule à chaque nouveau mot.

```
✅  $nomUtilisateur    $dateInscription    sePresenter()    calculerTotal()
❌  $nom_utilisateur   $DateInscription    SePresenter()    calculer_total()
```

Exemple complet

```php
<?php
class Voiture {

    public $marque;
```

```
    public $modele;
    public $couleur;

    public function demarrer() {
        return $this->marque . " " . $this->modele . " démarre.";
    }

    public function afficherInfo() {
        return $this->marque . " " . $this->modele . " (" . $this->couleur . ")";
    }
}
```

---

5. Instancier et utiliser un objet

Créer un objet

On utilise le mot-clé `new` suivi du nom de la classe :

```
$maVoiture = new Voiture();
```

À ce moment, PHP crée un nouvel objet de type `Voiture` et le stocke dans la variable `$maVoiture`.

Accéder aux propriétés et méthodes

On utilise l'opérateur `->` (tiret + chevron fermant) :

```
// Assigner des valeurs aux propriétés
$maVoiture->marque = "Peugeot";
$maVoiture->modele = "308";
$maVoiture->couleur = "bleue";

// Lire une propriété
echo $maVoiture->marque; // "Peugeot"

// Appeler une méthode
echo $maVoiture->demarrer(); // "Peugeot 308 démarre."
```

Plusieurs objets à partir d'une même classe

```
$v1 = new Voiture();
$v1->marque = "Peugeot";
$v1->modele = "308";

$v2 = new Voiture();
$v2->marque = "Renault";
$v2->modele = "Clio";

// Les deux objets sont totalement indépendants
echo $v1->marque; // "Peugeot"
echo $v2->marque; // "Renault"
```

> 📌 Modifier `$v1` n'affecte pas `$v2`. Chaque objet possède sa propre copie des propriétés.

6. Le mot-clé `$this`

Pourquoi `$this` existe

Quand une méthode s'exécute, elle a besoin de savoir **à quel objet elle appartient** pour accéder à ses propriétés. C'est le rôle de `$this`.

> `$this` est une référence automatique vers **l'objet courant** — celui qui est en train d'exécuter la méthode.

Analogie : Si je décris mes caractéristiques, je dis *"mon nom"*, *"mon âge"*. Le mot *"mon"* fait référence à moi. Dans une méthode PHP, `$this` joue exactement ce rôle.

Utilisation

```
class Personne {
    public $nom;
    public $age;

    public function sePresenter() {
        // $this->nom = "mon nom" (celui de cet objet)
        // $this->age = "mon âge" (celui de cet objet)
        return "Je suis " . $this->nom . ", " . $this->age . " ans.";
    }
}
```

Lorsqu'on appelle `$p->sePresenter()`, PHP remplace automatiquement `$this` par `$p` à l'intérieur de la méthode.

`$this` avec plusieurs objets

```
$p1 = new Personne();
$p1->nom = "Marie";

$p2 = new Personne();
$p2->nom = "Paul";

echo $p1->sePresenter();
// $this = $p1 → "Je suis Marie, ..."

echo $p2->sePresenter();
// $this = $p2 → "Je suis Paul, ..."
```

La même méthode produit un résultat différent selon l'objet qui l'appelle, parce que `$this` change.

`$this` pour appeler d'autres méthodes

On peut aussi utiliser `$this` pour appeler une autre méthode de la même classe :

```
class Etudiant {
    public $nom;
    public $notes = [];

    public function calculerMoyenne() {
        if (empty($this->notes)) return 0;
```

```
        return array_sum($this->notes) / count($this->notes);
    }

    public function afficherBilan() {
        $moyenne = $this->calculerMoyenne(); // ← appel via $this
        return $this->nom . " — Moyenne : " . $moyenne . "/20";
    }
}
```

---

7. Les méthodes en détail

Une méthode, c'est une fonction dans une classe

Une méthode se déclare exactement comme une fonction PHP, mais à l'intérieur de la classe. Elle peut :

- Prendre des paramètres
- Retourner une valeur
- Accéder aux propriétés de l'objet via `$this`
- Appeler d'autres méthodes de la classe via `$this`

Méthodes avec paramètres

```
class CompteBancaire {
    public $solde = 0;

    public function deposer($montant) {
        if ($montant > 0) {
            $this->solde += $montant;
            return "Dépôt effectué. Solde : " . $this->solde . "€";
        }
        return "Montant invalide.";
    }
}
```

Méthodes de calcul (propriétés calculées)

Certaines méthodes calculent une valeur à partir des propriétés sans la stocker — c'est une **propriété calculée** :

```
class Produit {
    public $prix    = 0;
    public $quantite = 0;

    // Pas de propriété $prixTTC stockée — calculée à la demande
    public function getPrixTTC() {
        return $this->prix * 1.20;
    }

    // Pas de propriété $valeurStock — calculée à la demande
    public function getValeurStock() {
        return $this->prix * $this->quantite;
```

```
    }
}
```

---

8. Erreurs courantes

Oublier `$this->`

```
class Personne {
    public $nom = "Marie";

    public function afficher() {
        return $nom;        // ❌ Variable locale inexistante
        return $this->nom;  // ✅ Propriété de l'objet
    }
}
```

Oublier `new`

```
$compte = CompteBancaire(); // ❌ Appel de fonction — erreur fatale
$compte = new CompteBancaire(); // ✅
```

Appeler une méthode sur la classe (pas sur un objet)

```
echo Voiture->demarrer(); // ❌ Erreur de syntaxe

$v = new Voiture();
echo $v->demarrer(); // ✅
```

Nom de classe mal écrit

```
class compteBancaire { }  // ❌ Pas de PascalCase
class CompteBancaire { }  // ✅
```

Propriété sans `$this` dans une méthode

```
class Compte {
    public $solde = 0;

    public function deposer($montant) {
        $solde += $montant;       // ❌ Modifie une variable locale
        $this->solde += $montant; // ✅ Modifie la propriété de l'objet
    }
}
```

---

9. Exercices corrigés

Exercice 1 — Classe `Produit`

```php
<?php
class Produit {
    public $nom;
    public $prix;
    public $stock;

    public function afficherInfo() {
        return $this->nom . " — " . $this->prix . "€ — Stock : " . $this->stock;
    }

    public function estDisponible() {
        return $this->stock > 0;
    }

    public function calculerPrixTTC() {
        return round($this->prix * 1.20, 2);
    }
}

// Tests
$p1 = new Produit();
$p1->nom   = "Ordinateur portable";
$p1->prix  = 899.99;
$p1->stock = 5;

echo $p1->afficherInfo();                                    // Ordinateur portable — 899.99
echo $p1->estDisponible() ? "Disponible" : "Indisponible"; // Disponible
echo $p1->calculerPrixTTC() . "€";                          // 1079.99€
```

Exercice 2 — Classe `CompteBancaire`

```php
<?php
class CompteBancaire {
    public $titulaire;
    public $solde = 0;

    public function deposer($montant) {
        if ($montant <= 0) return "Erreur : montant invalide.";
        $this->solde += $montant;
        return "Dépôt de " . $montant . "€. Solde : " . $this->solde . "€";
    }

    public function retirer($montant) {
        if ($montant <= 0)           return "Erreur : montant invalide.";
        if ($montant > $this->solde) return "Erreur : solde insuffisant.";
        $this->solde -= $montant;
        return "Retrait de " . $montant . "€. Solde : " . $this->solde . "€";
    }

    public function afficherSolde() {
        return $this->titulaire . " — Solde : " . $this->solde . "€";
    }
}
```

```php
// Tests
$c = new CompteBancaire();
$c->titulaire = "Marie Dupont";
echo $c->deposer(1000);   // Dépôt de 1000€. Solde : 1000€
echo $c->retirer(200);    // Retrait de 200€. Solde : 800€
echo $c->retirer(5000);   // Erreur : solde insuffisant.
echo $c->afficherSolde(); // Marie Dupont — Solde : 800€
```

Mini-projet — Gestion de stock

`Article.php`

```php
<?php
class Article {
    public $reference;
    public $nom;
    public $prix;
    public $quantite;

    public function ajouterStock($qte) {
        if ($qte <= 0) return "Erreur : quantité invalide.";
        $this->quantite += $qte;
        return $qte . " unité(s) ajoutée(s). Stock : " . $this->quantite;
    }

    public function retirerStock($qte) {
        if ($qte <= 0)            return "Erreur : quantité invalide.";
        if ($qte > $this->quantite) return "Erreur : stock insuffisant.";
        $this->quantite -= $qte;
        return $qte . " unité(s) retirée(s). Stock restant : " . $this->quantite;
    }

    public function estDisponible() {
        return $this->quantite > 0;
    }

    public function afficherDetails() {
        $statut = $this->estDisponible() ? "En stock" : "Rupture";
        return "[" . $this->reference . "] " . $this->nom
             . " — " . $this->prix . "€ — " . $this->quantite . " unité(s) (" . $statut . "
    }
}
```

`Client.php`

```php
<?php
class Client {
    public $nom;
    public $email;
    public $panier = [];

    public function ajouterAuPanier($article) {
        if (!$article->estDisponible()) {
            return "Erreur : '" . $article->nom . "' est en rupture de stock.";
```

```php
        }
        $this->panier[] = $article;
        return "'" . $article->nom . "' ajouté au panier.";
    }

    public function calculerTotal() {
        $total = 0;
        foreach ($this->panier as $article) {
            $total += $article->prix;
        }
        return $total;
    }

    public function afficherPanier() {
        if (empty($this->panier)) return "Le panier est vide.";
        $resultat = "Panier de " . $this->nom . " :\n";
        foreach ($this->panier as $i => $article) {
            $resultat .= ($i + 1) . ". " . $article->nom . " — " . $article->prix . "€\n";
        }
        $resultat .= "Total : " . $this->calculerTotal() . "€";
        return $resultat;
    }
}
```

---

10. Résumé

Les 5 notions clés de la séance

Notion	En une phrase
Objet	Une entité avec des données (propriétés) et des actions (méthodes)
Classe	Le plan qui décrit comment construire les objets
`new`	Crée un objet à partir d'une classe
`->`	Accède aux propriétés et méthodes d'un objet
`$this`	Référence l'objet courant depuis l'intérieur d'une méthode

Ce qui arrive en séance 2

- **La visibilité** — `public`, `private`, `protected` : qui a le droit d'accéder à quoi ?
- **Le constructeur** `__construct()` — initialiser automatiquement un objet à sa création
- **L'encapsulation** — protéger les données avec des getters et des setters

---

Ressources

- PHP.net — Classes et Objets
- PHP.net — Le mot-clé `$this`
- OpenClassrooms — POO en PHP

- PHP The Right Way — OOP